

Student 6 **H Harshavardhan** 192511171RC

2 / 2 submitted

Q1. Student Grade Book Application

CODE

```
def grade(avg):
    if avg >= 90:
        return 'O'
    elif avg >= 80:
        return 'A+'
    elif avg >= 70:
        return 'A'
    elif avg >= 60:
        return 'B+'
    elif avg >= 50:
        return 'B'
    else:
        return 'F'

n=int(input("enter the number of students:\n"))
students={}
for i in range(n):
    name=input("enter student name:\n")
    marks=list(map(int,input("enter 5 subject marks:\n").split()))
    students[name]=marks
result={}
    name:(
        sum(marks),
        sum(marks)/len(marks),
        grade(sum(marks)/len(marks))
    )
    for name,marks in students.items()
}
topper=max(result.items(),key=lambda x:x[1][1])
subjectavg=[]
for i in range(5):
    total=0
    for marks in students.values():
        total+=marks[i]
    subjectavg.append(total/n)
bestsubject=subjectavg.index(max(subjectavg))+1
sortedresult=sorted(result.items(),key=lambda x: x[1][1],reverse=True)
print("\nstudent report")
print("-"*50)
for name,data in sortedresult:
    total,avg,g=data
    print(f"{name:10} total={total:3}average={avg:.2f}grade={g}")
print("\nclass topper:",topper[0],f"average={topper[1][1]:.2f}")
print("subject with highest overall mean:subject",bestsubject)
```

INPUT

```
5
harsha
90 85 80 95 88
jaya
70 75 72 78 74
bala
95 96 94 92 98
sub
60 65 70 62 68
thowifq
82 84 81 80 83
```

OUTPUT

(no output)

Q2. Debugging and Rewriting

CODE

```
"""Bug1:Missing indentation for functional bodies
Bug2:wrong operator used in deposit functional(=+)
    correct operator is(+=)
Bug3:The condition balance (>) amount wrongly blocks withdraw the exact balance amount;
    it should be balance (≥) amount to allow full withdraw
Bug4: Incorrect slicing in mini_statement"""

"""DEBUGGED CODE"""
accounts={}
def create_account(acc_no,name,balance):
    accounts[acc_no]={'name':name,'balance':balance,'txns':[]}

def deposit(acc_no,amount):
    if acc_no in accounts:
        accounts[acc_no]['balance']+=amount
        accounts[acc_no]['txns'].append(('deposit',amount))
def withdraw(acc_no,amount):
    if accounts[acc_no]['balance']≥amount:
        accounts[acc_no]['balance']-=amount
        accounts[acc_no]['txns'].append(('withdraw',amount))
    else:
        print('insufficient balance')
def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5:]:
        print(t)
create_account(1001,'ravi',10000)
deposit(1001,5000)
withdraw(1001,20000)
withdraw(1001,3000)
mini_statement(1001)
print('balance:',accounts[1001]['balance'])
```

OUTPUT (no output)